Name : Piyush Gupta

Roll No. : 2401033

Div : T12

Subject : ASD & D

Experiment : 1

Aim : To perform version control operations using Git.

This experiments map to LO2.

LO 2 : Use git effectively for version control and source code management and collaborative team management.

Theory :

Git is a distributed version control system that helps developers track changes in source code, manage different versions of a project, and collaborate efficiently. It provides features such as repository management, branching, staging, committing, merging, and synchronization with remote repositories like GitHub.

1. **git clone <repository-url>**
   The git clone command creates a complete copy of a remote Git repository on your local computer. It downloads all project files, branches, and commit history so you can start working on the project.

2. **cd <repository-name>**
   The cd (Change Directory) command is used to move into the project folder. All Git commands should be executed from inside the repository directory.

3. **git status**
   The git status command displays the current state of the repository. It shows modified, staged, and untracked files, along with the current branch.

4. **git config --global user.name "Your Name"**
   This command sets your Git username globally on your computer. The username is recorded as the author of every commit you make.

5. **git config --global user.email "your@email.com"**
   This command configures the email address associated with your Git account. GitHub uses this email to identify the author of commits.

6. **git config --list**
   This command displays all the current Git configuration settings, including username, email, editor, and other preferences.

7. **git branch**
   The git branch command lists all local branches in the repository. The currently active branch is marked with an asterisk (*).

8. **git branch <branch-name>**
   This command creates a new branch without switching to it. Branches are used to develop features independently without affecting the main code.

9. **git checkout <branch-name>**
   The git checkout command switches from the current branch to an existing branch. Any new changes will now be made in that branch.

10. **git checkout -b <branch-name>**
    This command creates a new branch and immediately switches to it. It combines branch creation and checkout into a single command.

11. **git add <file-name>**
    The git add command stages a specific file for the next commit. Only staged files are included when a commit is created.

12. **git add .**
    This command stages all new, modified, and deleted files in the current directory. It prepares all changes for committing.

13. **git commit -m "Commit Message"**
    The git commit command saves all staged changes in the local repository. The commit message briefly describes the purpose of the changes.

14. **git log**
    The git log command displays the complete history of commits in the repository. It includes commit IDs, author names, dates, and messages.

15. **git log --oneline**
    This command displays the commit history in a compact format. Each commit is shown in a single line with a short commit ID and message.

16. **git diff**
    The git diff command compares the current working directory with the last committed version. It highlights all modifications that have not yet been staged.

17. **git diff origin/main**
    This command compares your local branch with the origin/main branch. It helps identify differences between local and remote repositories.

18. **git fetch origin**
    The git fetch command downloads the latest updates from the remote repository without merging them. It lets you review changes before integrating them.

19. **git pull origin main**
    The git pull command downloads and automatically merges the latest changes from the remote repository into the current branch. It keeps the local repository updated.

20. **git push origin main**
    The git push command uploads local commits to the remote GitHub repository. This makes your latest changes available to other collaborators.

21. **git restore --staged <file-name>**
    This command removes a file from the staging area while keeping its modifications in the working directory. It is useful when a file is staged by mistake.

22. **git merge <branch-name>**
    The git merge command combines changes from another branch into the current branch. It is commonly used to integrate completed features into the main branch.

23. **git branch -d <branch-name>**
    This command deletes a local branch after it has been successfully merged. It helps keep the repository clean and organized.

24. **git push origin --delete <branch-name>**
    This command removes a branch from the remote GitHub repository. It is usually performed after the branch has been merged and is no longer needed.

25. **git status**
    This final git status command verifies that all changes have been committed and pushed successfully. A clean working tree indicates there are no pending changes.

26. **git add index.html**
    This command stages the index.html file for the next commit. Git starts tracking the changes made to this file after it is added to the staging area.

27. **git commit -m "Add empty index.html file"**
    This command creates a new commit containing the staged index.html file. The commit message explains that an empty HTML file has been added.

28. **echo "<h1>Welcome to Agile Software 1</h1>" >> index.html**
    The echo command appends the given HTML text to the index.html file. This modifies the file, allowing Git to detect the changes.

29. **git diff**
    This command displays the changes made to index.html after adding the HTML heading. It allows you to review the modifications before staging and committing them.

Output :

```
(base) PS C:\Users\Lab203\Desktop\ASDLAB1> git --version
git version 2.48.1.windows.1
(base) PS C:\Users\Lab203\Desktop\ASDLAB1> git clone https://github.com/PiyushGupta25/AgileSoftware1.git
Cloning into 'AgileSoftware1'...
warning: You appear to have cloned an empty repository.
(base) PS C:\Users\Lab203\Desktop\ASDLAB1> cd AgileSoftware1
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git log
fatal: your current branch 'main' does not have any commits yet
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git branch feature1
fatal: not a valid object name: 'main'
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git branch -b  feature1
error: unknown switch `b'
usage: git branch [<options>] [-r | -a] [--merged] [--no-merged]
   or: git branch [<options>] [-f] [--recurse-submodules] <branch-name> [<start-point>]
   or: git branch [<options>] [-l] [<pattern>...]
   or: git branch [<options>] [-r] (-d | -D) <branch-name>...
   or: git branch [<options>] (-m | -M) [<old-branch>] <new-branch>
   or: git branch [<options>] (-c | -C) [<old-branch>] <new-branch>
   or: git branch [<options>] [-r | -a] [--points-at]
   or: git branch [<options>] [-r | -a] [--format]
```

```
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git checkout -b feature
Switched to a new branch 'feature'
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git origin main
git: 'origin' is not a git command. See 'git --help'.
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git config --global user.name "Piyushgupta25"
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git config --global user.email "piyushg9892@gmail.com"
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Piyushgupta25
user.email=piyushg9892@gmail.com
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git status
On branch feature

No commits yet
```

```
nothing to commit (create/copy files and use "git add" to track)
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> echo "# AgileSoftware1" >> README.md
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git add README.md
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git add .
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git commit -m "Initial commit: Add README.md"
[feature (root-commit) d1e22a6] Initial commit: Add README.md
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.md
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git remote -v
origin  https://github.com/PiyushGupta25/AgileSoftware1.git (fetch)
origin  https://github.com/PiyushGupta25/AgileSoftware1.git (push)
```

```
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git checkout main
>>
error: pathspec 'main' did not match any file(s) known to git
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git checkout -b main
>>
Switched to a new branch 'main'
Your branch is based on 'origin/main', but the upstream is gone.
  (use "git branch --unset-upstream" to fixup)
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git push -u origin main
remote: Permission to PiyushGupta25/AgileSoftware1.git denied to MonishaaDalwani.
fatal: unable to access 'https://github.com/PiyushGupta25/AgileSoftware1.git/': The requested URL returned error: 403
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git config --global user.email "piyushg9892@gmail.com"
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git config --global user.name "PiyushGupta25"
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git add .
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git commit -m "NEW CHanges"
On branch main
Your branch is based on 'origin/main', but the upstream is gone.
  (use "git branch --unset-upstream" to fixup)

nothing to commit, working tree clean
```

```
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git push origin main
>>
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 270 bytes | 270.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/PiyushGupta25/AgileSoftware1.git
 * [new branch]      main -> main
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git push origin main
Everything up-to-date
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git branch
  feature
* main
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git branch feature/project-setup
fatal: cannot lock ref 'refs/heads/feature/project-setup': 'refs/heads/feature' exists; cannot create 'refs/hea
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git branch feature1/project-setup
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git checkout feature1/project-setup
Switched to branch 'feature1/project-setup'
```

```
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git log
commit d1e22a63f69b3d606c731ccab930cfb02de69437 (HEAD -> feature1/project-setup, origin/main, main, feature)
Author: Piyushgupta25 <piyushg9892@gmail.com>
Date:   Tue Jul 14 05:19:08 2026 +0530

    Initial commit: Add README.md
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git diff
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git fetch origin
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git pull origin main
From https://github.com/PiyushGupta25/AgileSoftware1
 * branch            main       -> FETCH_HEAD
Already up to date.
```

```
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git status
On branch feature1/project-setup
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git add index.html
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git commit -m "Add empty index.html file"
[feature1/project-setup ef31196] Add empty index.html file
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 index.html
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> echo "<h1>Welcome to Agile Software 1</h1>" >> index.html
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git diff
diff --git a/index.html b/index.html
index e69de29..170d474 100644
Binary files a/index.html and b/index.html differ
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git push origin main
Everything up-to-date
```

```
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git mv index.html home.html
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git status
On branch feature1/project-setup
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    index.html -> home.html

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   home.html
```

```
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git rm -f home.html
>>
rm 'home.html'
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git status
On branch feature1/project-setup
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    index.html

(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git commit -m "Remove index.html"
>>
[feature1/project-setup e56f070] Remove index.html
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 index.html
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git status
On branch feature1/project-setup
nothing to commit, working tree clean
```

```
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git checkout main
>>
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git pull origin main
>>
From https://github.com/PiyushGupta25/AgileSoftware1
 * branch            main       -> FETCH_HEAD
Already up to date.
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git merge feature1/project-setup
>>
Updating d1e22a6..e56f070
Fast-forward
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git push origin main
>>
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 458 bytes | 458.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/PiyushGupta25/AgileSoftware1.git
   d1e22a6..e56f070  main -> main
(base) PS C:\Users\Lab203\Desktop\ASDLAB1\AgileSoftware1> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Conclusion :

In this experiment, I successfully performed various Git version control operations, including repository management, branching, staging, committing, comparing changes, and synchronizing with GitHub. This experiment helped me understand Git workflows, version

tracking, and collaborative software development, while improving my practical skills in managing software projects efficiently.